STATE MANAGEMENT - QUESTION PAPER

Comprehensive Assessment - Revised Focus Paper

Subject	State Management	**Total Marks**	100
Name	______________________	**Date**	______________________

Instructions

- Answer all questions in the space provided. Use additional sheets only if required.
- Read each question carefully. Avoid repeating the same idea in multiple answers.
- Section order in this paper is A, then C, then B.
- This paper gives lesser focus to framework-to-framework comparison and stronger focus to practical application of core concepts.
- Use the same clear font and neat presentation throughout your answers.

SECTION A - THEORY

Total: 30 Marks

A1) Multiple Choice Questions (16 x 1 mark = 16 marks)

1. (1 mark) State management is best defined as:

A. A way to colour components globally
B. The practice of deciding where data lives, how it changes, and how the UI stays in sync
C. A method used only for API retries
D. A replacement for CSS architecture

Answer: ________________________________

2. (1 mark) Which of the following is an example of UI / Local state?

A. Logged-in user role
B. Vendor assessment list from API
C. Whether one accordion is open
D. Notification data returned from server

Answer: ________________________________

3. (1 mark) According to the decision tree, if data is shared across components and comes from an API, it should be treated as:

A. Local component state
B. Shared client state only
C. Server / remote state
D. Static hard-coded data

Answer: ________________________________

4. (1 mark) The Golden Rule says:

A. Everything should go into one global store
B. Local UI state must stay local unless multiple disconnected components truly need it
C. All server state belongs in component state
D. Only session data matters in state management

Answer: ________________________________

5. (1 mark) What is the correct unidirectional flow?

A. Store -> UI -> action -> handler -> store update -> UI re-render
B. UI -> store -> DOM -> API -> handler
C. Handler -> action -> store -> UI
D. API -> UI -> action -> store

Answer: ________________________________

6. (1 mark) Which problem is most directly reduced by proper state management?

A. Browser installation issues
B. UI inconsistency across screens
C. Font loading delay
D. Image compression errors

Answer: ________________________________

7. (1 mark) In a cache-first strategy, the first preferred step on a repeat visit is to:

A. Always show a full-page spinner
B. Clear all old data first
C. Render cached data immediately if available
D. Block the screen until the network responds

Answer: ________________________________

8. (1 mark) TTL stands for:

A. Total Transfer Layer

B. Time To Live

C. Time To Load

D. Table Type Limit

Answer: ________________________________

9. (1 mark) If cached data is stale but still available, the correct behaviour is to:

A. Remove the data from the UI and show white space

B. Show the existing data and revalidate in the background

C. Force logout

D. Wait for a manual refresh only

Answer: ________________________________

10. (1 mark) Optimistic UI means:

A. The UI waits for the server before changing

B. The UI updates immediately assuming success, then rolls back if needed

C. The UI skips API calls

D. The UI stores everything in localStorage

Answer: ________________________________

11. (1 mark) Which action is a poor candidate for optimistic UI?

A. Pinning a risk card

B. Marking a notification as read

C. Reordering low-stakes list items

D. Submitting a regulatory compliance filing

Answer: ________________________________

12. (1 mark) Every async operation must be designed for:

A. Loading, success, and error

B. Start and stop only

C. Cache and refetch only

D. Request and response only

Answer: ________________________________

13. (1 mark) For an initial page-level load, the preferred UX is:

A. A blank white page

B. A generic spinner in the corner

C. A skeleton screen that mirrors the final layout

D. No visual feedback

Answer: ________________________________

14. (1 mark) A presentational component should:

A. Fetch data directly from the API

B. Know store structure and dispatch actions itself

C. Receive props and emit events upward

D. Hold all application state

Answer: ________________________________

15. (1 mark) Memoisation primarily helps by:

A. Preventing all network requests

B. Avoiding unnecessary recomputation and re-renders when inputs do not change
C. Replacing selectors and getters completely
D. Storing data permanently in cache

Answer: ______________________________

16. (1 mark) Normalised state is preferred because it:

A. Duplicates data to improve readability
B. Makes updates slower but simpler
C. Stores entities in flat structures keyed by ID and avoids duplication bugs
D. Removes the need for selectors or getters

Answer: ______________________________

A2) Short Written Answers (7 x 2 marks = 14 marks)

17. (2 marks) Why does state management become essential as an enterprise application grows from a few screens to many modules, roles, and API calls?

____

____

____

____

18. (2 marks) Explain the three types of state - Local / UI, Shared / Global, and Server / Remote - with one clear example for each.

____

____

____

____

____

19. (2 marks) Write the decision tree you would use to decide where a new piece of data should live.

____

____

____

____

____

20. (2 marks) Why is storing derived values in state risky? Give one simple example using a risk count, total, or status summary.

____

21. (2 marks) Explain cache-first data strategy in your own words. Include both freshness checking and background revalidation.

22. (2 marks) State the rule for skeleton screens versus spinners. When should each be used?

23. (2 marks) What duplication bug does state normalisation prevent when the same user or owner object appears inside many records?

SECTION C - SCENARIO-BASED WRITING AND PSEUDOCODE

Total: 20 Marks

C1) Scenario Writing Questions (5 x 2 marks = 10 marks)

24. (2 marks) A navbar badge shows 5 open risks, but the detailed table shows 4. Explain what kind of state problem this indicates and how a single source of truth would fix it.

25. (2 marks) Users returning to a list page always see a blank white screen for two seconds before data appears, even when they visited the page just a minute ago. Identify the design rules being violated and describe the correct behaviour.

26. (2 marks) Your team is debating whether to use optimistic UI for a 'Submit compliance audit' action. Explain whether it should be optimistic or pessimistic, and why.

27. (2 marks) A developer places `isAccordionOpen`, the selected tab index, and one modal input field into the global store. Explain two harms this causes and classify the data correctly.

28. (2 marks) A parent dashboard component re-renders often, and a child table with many rows becomes slow even though the row props rarely change. Explain where memoisation can help and why.

C2) Scenario Pseudocode Questions (5 x 2 marks = 10 marks)

29. (2 marks) Write language-agnostic pseudocode for `fetchRisks()` using a cache-first pattern with a TTL check. It should use cached data immediately when fresh, and perform background revalidation when stale.

30. (2 marks) Write pseudocode for `approveRisk(riskId)` using the five-step optimistic pattern: snapshot, immediate update, API call, success handling, and rollback on failure.

31. (2 marks) Write pseudocode for a component render handler that supports all three async states correctly: loading with skeleton, success with data, and error with retry button.

32. (2 marks) Write pseudocode for a function chooseStatePlacement(item) that follows the decision tree and returns whether the item belongs in Local / UI state, Shared / Global state, or Server / Remote state.

33. (2 marks) Write pseudocode to normalise an array of risk records into { byId, allIds } and show how a related ownerId reference is stored instead of duplicating the full owner object.

SECTION B - PRODUCT APPLICATION

Total: 50 Marks

Answers in this section must be written using one real product or module you know well. Generic answers will get fewer marks.

B1) Product Application Multiple Choice Questions (10 x 1 mark = 10 marks)

34. (1 mark) In your product, a filter modal has a temporary search input that is used only inside that modal while the user types. The best place for this state is:

A. Local component state
B. Shared global store by default
C. Server cache
D. Hard-coded constants

Answer: ______________________________

35. (1 mark) A risk register list appears in the dashboard, summary widget, and detail page, and it comes from the backend. The most correct classification is:

A. Local UI state
B. Shared user preference
C. Server / remote state
D. Static theme state

Answer: ______________________________

36. (1 mark) A user revisits a vendor list page within the chosen TTL window. The best behaviour is:

A. Show cached data immediately without blocking the user
B. Show a blank page until the API returns
C. Delete the cache and refetch
D. Show an error by default

Answer: ______________________________

37. (1 mark) A user revisits the same page after the data has become stale. The best behaviour is:

A. Hide the old data and freeze the screen
B. Show cached data, then revalidate in the background
C. Log the user out
D. Force a full browser reload

Answer: ______________________________

38. (1 mark) Which action in a business product is usually a good candidate for optimistic UI?

A. Pin or unpin a dashboard card
B. Final regulatory submission
C. Permanent record deletion
D. Password reset

Answer: ______________________________

39. (1 mark) Which action should usually remain pessimistic?

A. Mark as read
B. Expand or collapse a panel
C. Submit a compliance sign-off or audit submission
D. Toggle a low-risk preference

Answer: ______________________________

40. (1 mark) For the first load of a large table page, the best loading design is:

A. A small spinner in the top-right corner only
B. A skeleton with placeholder rows and column shapes that resemble the final table
C. No loading state at all
D. A plain text message saying please wait

Answer: ________________________________

41. (1 mark) A component both fetches API data and renders reusable row UI in the same place. This most directly violates:

A. Memoisation principle
B. Smart vs presentational separation
C. TTL calculation
D. State normalisation

Answer: ________________________________

42. (1 mark) A memoised derived value should recompute only when:

A. Any unrelated part of the page changes
B. The specific dependencies used to derive it change
C. The route changes, no matter what
D. A spinner disappears

Answer: ________________________________

43. (1 mark) When owner details are copied inside every risk record, control record, and task record, the main danger is:

A. Faster updates
B. Better caching
C. Data duplication leading to inconsistency when one copy changes and others do not
D. Automatic memoisation

Answer: ________________________________

B2) Product Application Written Questions (8 x 5 marks = 40 marks)

44. (5 marks) Choose one real module from your product. Create a state inventory with 8 items. For each item, write its type, where it should live, and one failure mode if placed incorrectly.

Item name	Type (Local / Shared / Server)	Where it lives	Why	Failure mode if placed incorrectly
1.				
2.				
3.				
4.				
5.				
6.				
7.				
8.				

45. (5 marks) Using the decision tree, classify 10 different state items from one real screen in your product. Your answer must include at least: one local state item, one shared state item, one server state item, one item that would be a mistake to keep globally, and one item whose placement affects security or permissions.

- Write the screen name first.

- Then list 10 state items and justify the classification of each.

46. (5 marks) Choose one list or detail page from your product and design a cache-first plan for it.

- State the data type and your chosen TTL.
- Explain what happens when the page is reopened within TTL.
- Explain what happens when the data is stale.
- Explain how the UI communicates freshness without blocking the user.

47. (5 marks) Choose one important API-driven page in your product and specify all three async states for that page.

- Loading - describe the exact skeleton layout and why it fits the final UI.
- Success - explain what fills each major area of the page.
- Error - write the message, fallback action, and retry behaviour.

48. (5 marks) Choose one reversible, low-stakes action in your product and design an optimistic UI flow for it.

- State the action.
- Write the five steps: snapshot, immediate update, background API call, success, failure rollback.
- Then name one high-stakes action in the same product that must not use optimistic UI, with reason.

49. (5 marks) Identify one component in your current product that mixes data logic and UI rendering too tightly.

- Explain what it currently does wrong.
- Split it into a smart/container component and a presentational component.
- List the props and events the presentational component should expose.

50. (5 marks) Perform a memoisation audit on one real product screen.

- Identify two places where unnecessary re-renders are likely happening.
- For each one, explain why it re-renders.
- State what should be memoised and what dependency or props should control recomputation.

51. (5 marks) Choose one list in your product where related user / owner / entity data is repeated inside many records.

- Show the current unnormalised shape briefly.
- Show the proposed normalised shape using IDs and flat maps.
- Explain what update bug this prevents and how it improves lookup or rendering efficiency.

FINAL MARK SUMMARY

Section	Component	Marks
A - Theory	16 MCQs + 7 short answers	30
C - Scenario Writing and Pseudocode	5 writing + 5 pseudocode	20
B - Product Application	10 MCQs + 8 written application questions	50

Grand Total: 100 Marks

Coverage note: This paper intentionally reduces direct framework comparison and instead emphasises practical understanding of state management, the decision tree, cache-first strategy, staleness and TTL, optimistic UI, async states, skeleton screens vs spinners, smart vs presentational components, memoisation, and state normalisation across all sections, especially Section B.